

TP3 : ARCHITECTURE DISTRIBUÉE

Microservices, Load Balancing & Cache System

Haytham Kennouz

Encadré par : Pr.ELBAGHAZAOUI

4 mars 2026

1 Introduction

L'évolution vers des architectures distribuées répond à un besoin critique de passage à l'échelle et de tolérance aux pannes. Ce rapport documente le déploiement d'un système robuste exploitant **Nginx** pour la répartition, **Redis** pour la performance, et **RabbitMQ** pour l'asynchronisme.

2 État du Déploiement

L'infrastructure complète a été conteneurisée via Docker. Les 7 services (Nginx, App1, App2, MongoDB, Redis, RabbitMQ et Consumer) sont isolés et communicants.

Visualisation de l'infrastructure

Logicielle\tp3 : monolith distribuee → (main) 0ms 3:17 PM

hk >> docker-compose -p tp3_monolith ps false

Name	Command	State	Ports
app1	docker-entrypoint.sh node	Up	0.0.0.0:3001->3001/tcp, :::3001->3001/tcp
app2	docker-entrypoint.sh node	Up	0.0.0.0:3002->3002/tcp, :::3002->3002/tcp
consumer	docker-entrypoint.sh node	Up	
mongodb	docker-entrypoint.sh mongod	Up	0.0.0.0:27017->27017/tcp, :::27017->27017/tcp
nginx	/docker-entrypoint.sh nginx	Up	0.0.0.0:80->80/tcp, :::80->80/tcp
rabbitmq	docker-entrypoint.sh rabbitmq	Up	15671/tcp, 0.0.0.0:15672->15672/tcp, :::15672->15672/tcp, 15691/tcp, 15692/tcp, 25672/tcp, 4369/tcp, 5671/tcp, 0.0.0.0:5672->5672/tcp, :::5672->5672/tcp
redis	docker-entrypoint.sh redis	Up	0.0.0.0:6379->6379/tcp, :::6379->6379/tcp

Logicielle\tp3 : monolith distribuee → (main) 755ms 3:17 PM

hk >> false

Figure 1 : Orchestration Docker Compose réussie.

3 Optimisation & Performance

3.1 Load Balancing & Round-Robin

Le trafic entrant est réparti dynamiquement entre les deux instances applicatives. En cas de défaillance d'une instance, la continuité du service est assurée de manière transparente.

3.2 Accélération par Cache Redis

L'implémentation d'un cache distribué permet de réduire drastiquement la charge sur MongoDB tout en améliorant le temps de réponse client.

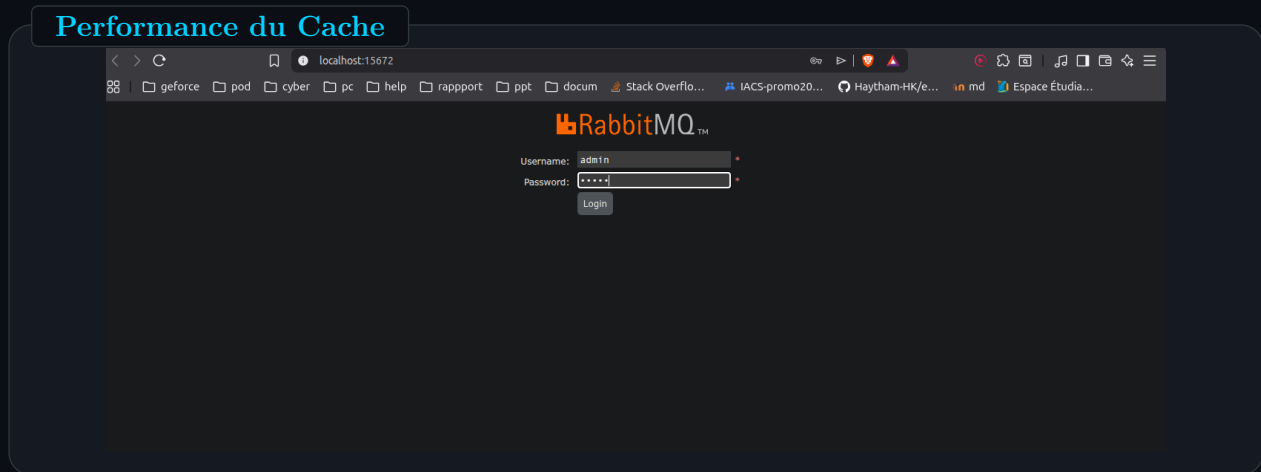


Figure 2 : Démonstration de l'alternance des instances et de la source de données (Redis vs DB).

4 Flux de Données & Validation

4.1 Traitement Synchrone (API REST)

La création de commandes est validée et stockée en base avant de déclencher les événements asynchrones.

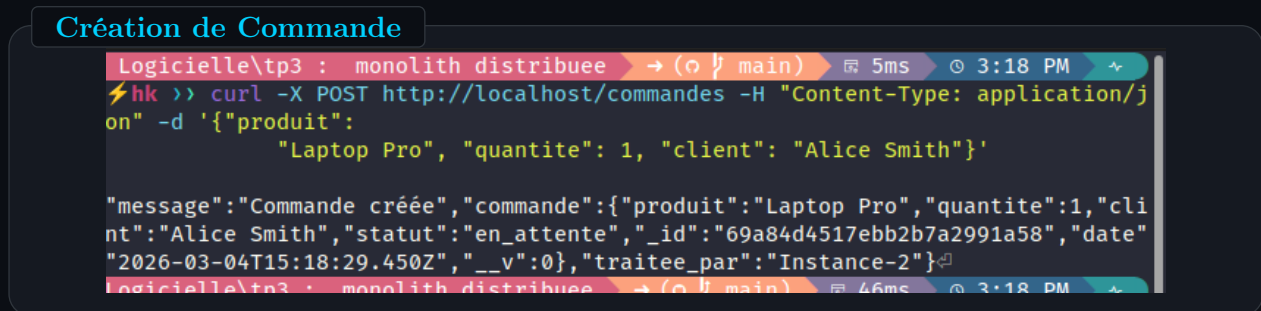


Figure 3 : Confirmation JSON d'une nouvelle commande via POST.

4.2 Traitement Asynchrone (RabbitMQ)

Le bus de messages permet de déléguer les tâches lourdes (notifications) à un service tiers, libérant instantanément le serveur applicatif.

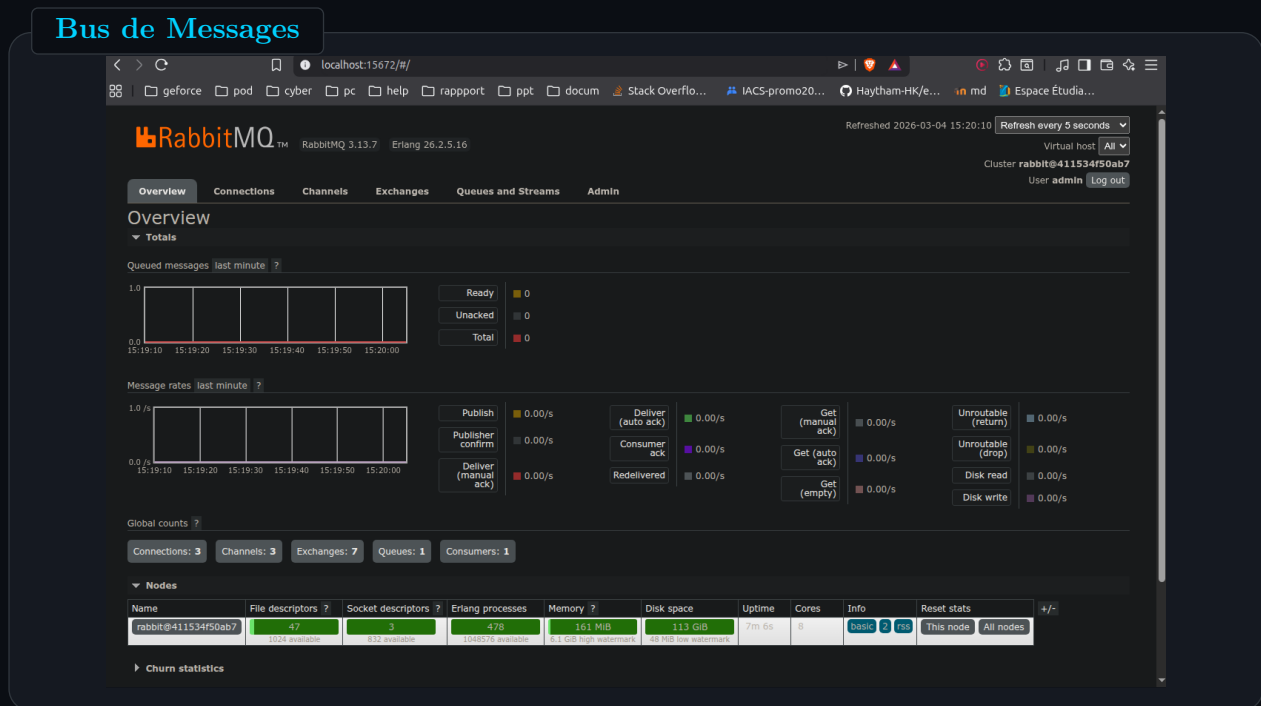


Figure 4 : Capture des logs du consommateur traitant les nouveaux messages.

5 Réponses aux Questions de Compte-rendu

Q1 : Quel composant reçoit en premier les requêtes du client ? Quel algorithme utilise-t-il ?

Le composant qui reçoit les requêtes en premier est le **Load Balancer (Nginx)**. Il utilise par défaut l'algorithme **Round-robin** pour répartir équitablement le trafic entre les instances applicatives disponibles.

Q2 : Que se passe-t-il si app1 tombe en panne ? Nginx continue-t-il à fonctionner ? Pourquoi ?

Si app1 tombe en panne, Nginx continue de fonctionner et redirige automatiquement les requêtes vers les instances saines (comme app2). Cela est possible car Nginx est un service indépendant qui gère la haute disponibilité en isolant les défaillances des nœuds applicatifs.

Q3 : Expliquez la différence observée entre le 1er GET (source MongoDB) et le 2ème GET (source Redis).

Le 1er GET est plus lent car il doit interroger **MongoDB** (stockage persistant sur disque). Le résultat est ensuite stocké en mémoire vive dans **Redis**. Le 2ème GET est quasi instantané car il récupère les données directement depuis le cache Redis, évitant ainsi un accès coûteux à la base de données.

Q4 : Pourquoi invalide-t-on le cache Redis après un POST /commande ?

On invalide le cache pour garantir la **cohérence des données**. Comme un POST modifie l'état de la base de données, les données précédemment mises en cache deviennent obsolètes. Supprimer l'entrée dans Redis force le prochain GET à récupérer les informations à jour.

Q5 : En quoi le bus de messages RabbitMQ améliore-t-il la résilience du système ?

RabbitMQ permet un **découplage asynchrone**. Si le microservice de notification (consommateur) subit une panne, les messages ne sont pas perdus ; ils sont stockés en sécurité dans la file d'attente et seront traités dès que le service redeviendra disponible, sans interrompre le flux principal de création de commandes.

Q6 : Si on voulait ajouter une 3ème instance Node.js, quelle ligne faudrait-il modifier dans nginx.conf ?

Il faudrait ajouter la ligne `server app3:3003;` à l'intérieur du bloc `upstream backend { ... }` dans le fichier `nginx/nginx.conf`.

6 Conclusion

Le système réalisé démontre une architecture moderne prête pour la production. L'interconnexion fluide entre la base de données persistante, le cache rapide et le bus de messages asynchrone constitue une base solide pour toute application web scalable.